Version Control

Date: 16/09/2026 Name: Manas Doshi

Think of version control as a time machine for your work. Here is a breakdown of what it is, why it matters, how it works, and the essential commands you need to get started.

1. What is Version Control?

Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later. A proper version control system automates this entire process, tracking every modification, who made it, and why.

2. Why is Version Control Important?

Without version control, working on projects—especially with a team—can get chaotic very fast. Here is why it is essential:

- **Undo Disasters:** If an update breaks your project, you can instantly revert back to a working state.
- **Complete History:** You get a full timeline showing who changed what, when, and why.
- **Safe Experimentation:** You can create separate workspace copies (branches) to test new ideas without risking the main working product.
- **Seamless Teamwork:** Multiple people can work on the exact same project simultaneously without overwriting each other's work.

3. Types of Version Control Systems

Version control systems generally fall into three categories:

Local Version Control Systems

- **How it works:** All versions of your files are stored in a database directly on your local computer.
- **The catch:** If your hard drive crashes or gets corrupted, you lose the entire project history. It also makes collaborating with others nearly impossible.

Centralized Version Control Systems (CVCS)

- **How it works:** A single central server holds all the project files and version history. Developers check out files from this central location onto their computers.
- **Examples:** Subversion (SVN), Perforce.
- **The catch:** If the central server goes down, nobody can save new versions or collaborate until it's back up.

Distributed Version Control Systems (DVCS)

- **How it works:** Every developer clones the entire repository onto their local machine, including its full version history. Everyone has a full backup of the project.
- **Examples:** Git, Mercurial.
- **Why it wins:** You can work offline, it is super fast, and if the main server crashes, any developer's local copy can restore it completely.

4. Essential Git Commands

Git is the most popular distributed version control system today. Here are the core commands you will use daily:

Command	What it Does
git init	**Starts a project.** Turns the current folder into a brand-new Git repository.
git clone <url>	**Downloads an existing project.** Copies a repository hosted remotely (e.g., on GitHub) to your machine.
git status	**Checks current state.** Shows which files have been modified, staged, or untracked.
git add <file>	**Stages changes.** Tells Git to start tracking specific files for the next save snapshot. Use git add . to stage all changes.
git commit -m "message"	**Saves a snapshot.** Captures a permanent record of staged changes with a descriptive message explaining what you did.
git branch <name>	**Creates a new line of development.** Lets you work on a feature in isolation without affecting the main codebase.

git checkout <branch>	**Switches work environments.** Moves your current workspace to a specified branch (or use git switch <branch>).
git merge <branch>	**Combines work.** Takes changes from one branch and merges them into your active branch.
git pull	**Fetches and updates.** Downloads the latest changes from a remote repository and merges them into your local files.
git push	**Uploads work.** Sends your local commits to a remote repository so others can see them.